



---

# Islamic University of Technology

Department of Computer Science and Engineering (CSE)

**CSE 4206:** Digital Logic Design Lab  
Summer, 2024-2025

---

Aashnan Rahman, Jr. Lecturer, CSE  
Faisal Hussain, Assistant Professor, CSE

## **Lab 1**

Introduction to Logic Gates and Their Truth Tables

# Contents

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                     | <b>2</b>  |
| <b>2</b> | <b>Getting Familiar with Today's Lab Equipment</b>                      | <b>3</b>  |
| 2.1      | Breadboard . . . . .                                                    | 3         |
| 2.2      | Integrated Circuits (ICs) . . . . .                                     | 4         |
| <b>3</b> | <b>General Working Procedure</b>                                        | <b>5</b>  |
| <b>4</b> | <b>Logic Gates</b>                                                      | <b>7</b>  |
| 4.1      | Basic Gates . . . . .                                                   | 7         |
| 4.1.1    | AND Gate . . . . .                                                      | 7         |
| 4.1.2    | OR Gate . . . . .                                                       | 7         |
| 4.1.3    | NOT Gate . . . . .                                                      | 8         |
| 4.2      | Universal Gates . . . . .                                               | 8         |
| 4.2.1    | NAND Gate . . . . .                                                     | 8         |
| 4.2.2    | NAND Gate . . . . .                                                     | 9         |
| 4.2.3    | NOR Gate . . . . .                                                      | 9         |
| 4.3      | Exclusive Gates . . . . .                                               | 9         |
| 4.3.1    | X-OR . . . . .                                                          | 10        |
| 4.3.2    | X-NOR . . . . .                                                         | 10        |
| <b>5</b> | <b>Overview of Logic Gates and ICs</b>                                  | <b>10</b> |
| <b>6</b> | <b>IC Internal Diagram</b>                                              | <b>11</b> |
| <b>7</b> | <b>Experiment 1: Introduction to Logic Gates and Their Truth Tables</b> | <b>12</b> |
| 7.1      | Objectives . . . . .                                                    | 12        |
| 7.2      | Tasks . . . . .                                                         | 12        |
| 7.2.1    | Task 1 . . . . .                                                        | 13        |
| 7.2.2    | Task 2 . . . . .                                                        | 14        |
| 7.2.3    | Task 3: Elevator Call System . . . . .                                  | 14        |
| <b>8</b> | <b>Additional Task (Installing Proteus)</b>                             | <b>15</b> |

# 1 Introduction

Digital Logic Design is a core subject in electronics and computer engineering that serves as the gateway to understanding how digital systems operate. It focuses on the use of binary logic (0s and 1s) to build circuits that perform decision-making and data-handling operations. This laboratory manual is structured to provide a comprehensive, hands-on learning experience that complements theoretical instruction and develops practical skills in digital circuit design.

The lab begins with fundamental experiments on logic gates, the basic building blocks of all digital systems. Students will learn how to implement and verify the behavior of these gates using truth tables and simple circuits. Building upon this, the course progresses through key combinational circuits such as adders, encoders, decoders, and multiplexers (MUX), introducing how logic can be used to perform arithmetic and data routing tasks.

The second phase of the lab explores sequential circuits, where students will work with flip-flops, counters, and registers—essential components that form the memory and control elements of a digital system. Each experiment is designed to strengthen the understanding of timing, state changes, and circuit stability.

In addition to traditional breadboard-based prototyping, the course introduces students to simulation platforms such as Proteus and TinkerCAD. These tools allow for circuit simulation, testing, and debugging in a virtual environment, offering valuable exposure to industry-standard design practices.

Toward the end of the lab, students will apply their accumulated knowledge in a mini-project, encouraging creativity and system-level thinking. By completing this lab course, students will be equipped with the foundational skills needed for advanced digital electronics, embedded systems, and computer architecture.

## 2 Getting Familiar with Today's Lab Equipment

In this lab, the primary apparatus that will be used is the Digital Logic Trainer Kit. This kit includes a solderless breadboard for circuit construction. The trainer kit also features a regulated power supply to provide the necessary voltage for the circuits. For input and output operations, the kit includes switches to provide logical inputs and LEDs to indicate circuit outputs. The trainer kit is designed to support a variety of digital logic experiments, from basic gate implementations to more complex combinational and sequential circuits. This setup provides a hands-on platform for students to learn, experiment, and troubleshoot digital circuits in a controlled environment.

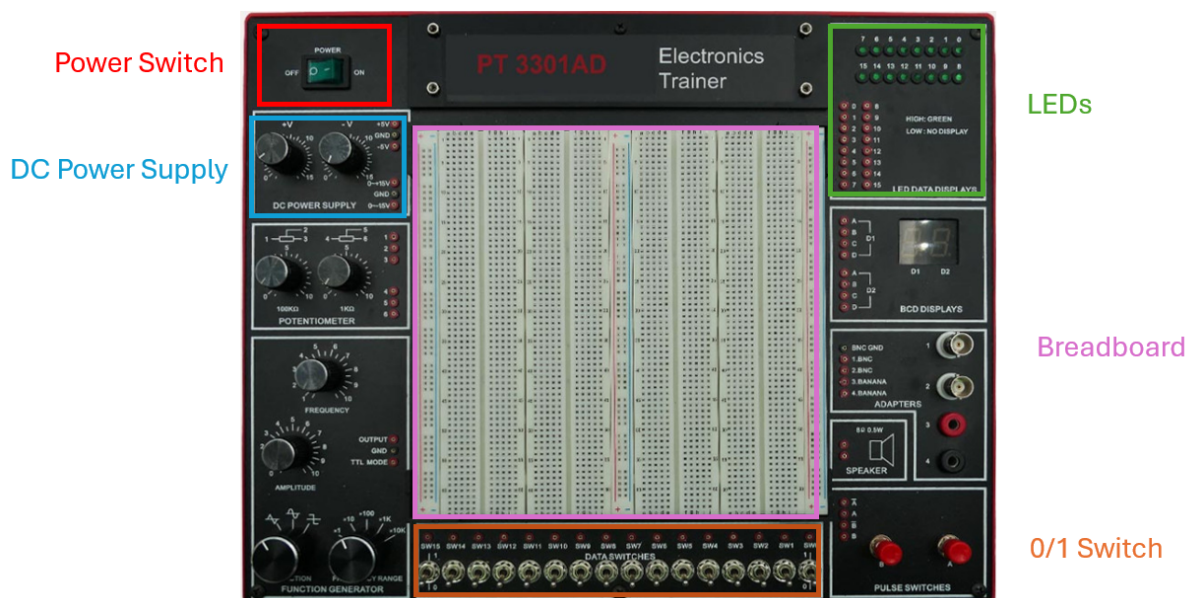


Figure 1: Analog Digital Trainer Board

### 2.1 Breadboard

The breadboard is an essential tool for constructing and testing digital circuits in this lab. It consists of two terminal strips and two bus strips (often broken in the center). Each bus strip has two rows of contacts, and each row forms a node, meaning all the contacts within a row are electrically connected. These bus strips are mainly used for power supply connections, but can also serve any node that requires a large number of connections.

The terminal strips on the breadboard are where the actual circuits are built. Each terminal strip has multiple rows and columns of contacts, with each row of contacts forming a single node. This design allows multiple components to be connected either in parallel or series.

It is important to ensure that the 5V power supply is **NOT EXCEEDED**, as doing so could damage the Integrated Circuits (ICs) used in the experiments. Incorrectly connecting power to the ICs could result in them overheating, malfunctioning, or even being physically damaged.

When constructing circuits, you will insert the leads of components into the contact receptacles on the terminal strips and make connections using wires. It is good practice to wire the +5V and 0V power supply connections to separate bus strips.

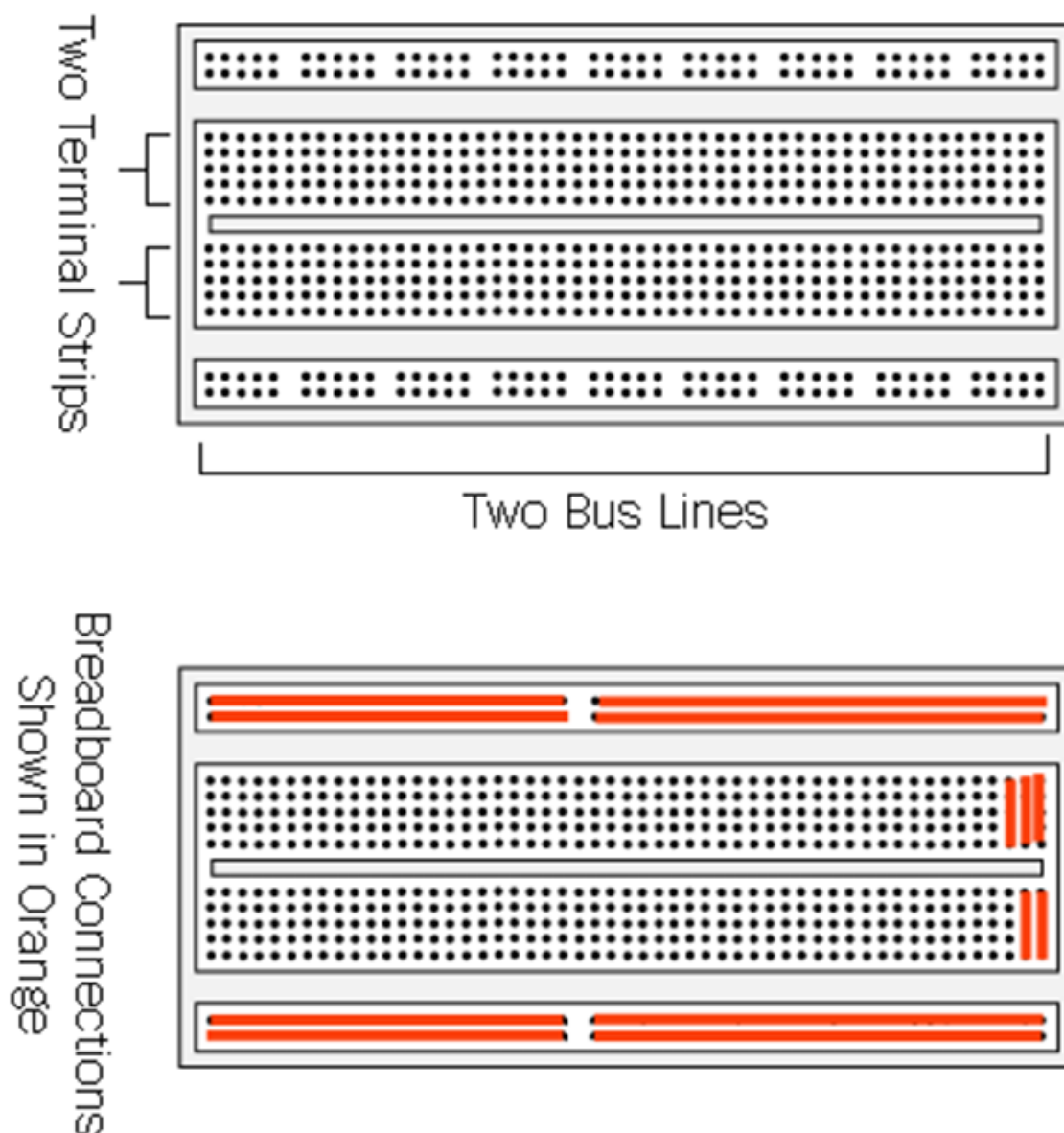


Figure 2: Breadboard

## 2.2 Integrated Circuits (ICs)

Digital Integrated Circuits (ICs) are a collection of **resistors**, **diodes**, and **transistors** fabricated on a single piece of semiconductor material, typically **silicon**, and are commonly referred to as a "**chip**." These chips are enclosed in a protective **plastic** or **ceramic** package, with pins extended out for easy connections to other devices and components.

One of the most common IC package types is the **Dual-In-Line Package (DIP)**, as shown in Figure 3. The pins of the IC are numbered **counterclockwise** when viewed from the top of the package, with a distinctive **notch** or **dot** indicating the starting point. A typical example of a DIP is a **14-pin package**, although ICs are also available in other pin configurations such as **16**, **20**, **24**, **28**, **40**, and **64** pins, depending on the complexity and functionality of the IC.

The **resistors**, **diodes**, and **transistors** embedded within the chip are configured to



Figure 3: Dual-in-Line Package (DIP))

perform specific logic operations and are collectively known as **logic gates**. These logic gates form the core of digital ICs and can vary in number depending on the specific IC.

Digital ICs are often categorized based on their **circuit complexity**, which is typically measured by the **number of equivalent logic gates** embedded within the chip. The complexity level can range from simple gates (like AND, OR, NOT) to more complex circuits involving flip-flops, multiplexers, and other combinational or sequential logic elements. Below is a table that categorizes ICs based on their complexity levels:

| Complexity Level                            | Description                                                                               |
|---------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>SSI</b> (Small Scale Integration)        | Contains a small number of gates (e.g., AND, OR).                                         |
| <b>MSI</b> (Medium Scale Integration)       | Contains dozens of gates, often forming more complex combinational logic.                 |
| <b>LSI</b> (Large Scale Integration)        | Contains hundreds of gates, enabling complex functions like memory and control.           |
| <b>VLSI</b> (Very Large Scale Integration)  | Contains thousands of gates, used in microprocessors, large memory chips, etc.            |
| <b>ULSI</b> (Ultra-Large Scale Integration) | Contains millions of gates, typically used in advanced processors and integrated systems. |

Table 1: IC Complexity Levels

This categorization helps in understanding the functionality and potential applications of different ICs, ranging from simple logic gates to highly complex, multifunctional chips.

### 3 General Working Procedure

These steps should be followed as the general procedure for today's experiment.

1. Always ensure that the power supply is turned **off** before building or modifying any part of your circuit.
2. Connect the **+5V** and **ground (GND)** outputs of the power supply to the corresponding power and ground bus strips on your breadboard.
3. Insert the required ICs into the breadboard. All chips should be oriented in the same direction, with **Pin 1** positioned at the upper-left corner. Pin 1 is usually marked by a small dot or notch on the chip package.

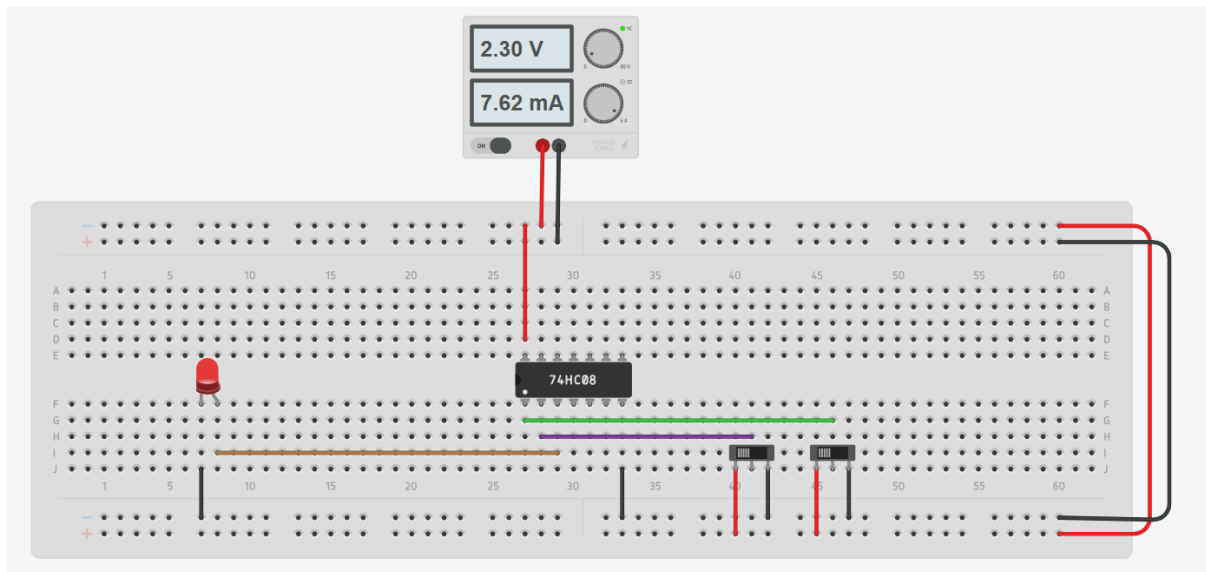


Figure 4: Setting up the breadboard

4. Connect the **+5V** and **GND** pins of each chip to the power and ground rails of the breadboard to ensure proper functionality.
5. Begin wiring the circuit according to your schematic. Use wire to make connections between the relevant pins of the chips. It is advisable to start with shorter connections before moving on to longer ones. As each connection is completed, mark it on your schematic to avoid duplication or errors.
6. If you discover a wiring error after applying power, **immediately turn off the power** before making any corrections or rewiring the circuit.

## 4 Logic Gates

In this section, we will discuss the different types of logic gates, their functions, and their corresponding truth tables.

### 4.1 Basic Gates

Basic logic gates are the fundamental components of digital electronics, used to perform simple logical operations on binary inputs. The three primary gates — **AND**, **OR**, and **NOT** serve as the building blocks for more complex digital circuits. Each gate follows a specific rule for processing input signals and producing an output. These gates are essential to understanding digital logic design. We will explore their behavior, functions, and truth tables in detail in the sections that follow.

#### 4.1.1 AND Gate

The AND gate is one of the most basic logic gates used in digital electronics. It produces a high output (logic 1) only when all of its inputs are high; if any input is low (logic 0), the output will also be low. It is commonly represented by a D-shaped symbol, with two or more input lines and a single output line. This gate is useful in situations where multiple conditions must be met at the same time for an action to occur.



Figure 5: 2 input AND Gate

| A | B | Output |
|---|---|--------|
| 0 | 0 | 0      |
| 0 | 1 | 0      |
| 1 | 0 | 0      |
| 1 | 1 | 1      |

Table 2: Truth Table of AND Gate

#### 4.1.2 OR Gate

The OR gate is a basic digital logic gate that outputs a high signal (logic 1) if at least one of its inputs is high. If all inputs are low (logic 0), only then will the output be low. This gate is used in situations where an action should occur when any one of multiple conditions is true. The OR gate is symbolized by a curved shape with two or more input lines and one output line. It is widely used in digital systems for decision-making processes where multiple inputs can independently trigger an outcome.



Figure 6: 2 input OR Gate

| A | B | Output |
|---|---|--------|
| 0 | 0 | 0      |
| 0 | 1 | 1      |
| 1 | 0 | 1      |
| 1 | 1 | 1      |

Table 3: Truth Table of OR Gate

### 4.1.3 NOT Gate

The NOT gate is the simplest type of logic gate, operating with only a single input and a single output. It performs a logical inversion, meaning it outputs the opposite of the input signal. If the input is high (logic 1), the output is low (logic 0), and vice versa. This gate is essential in digital logic for creating complementary signals and is commonly used to control logic flow in circuits. The symbol for the NOT gate is a triangle pointing to the right with a small circle (representing inversion) at its output. Although simple, the NOT gate plays a critical role in building complex digital systems by enabling the creation of control and timing signals through signal negation.

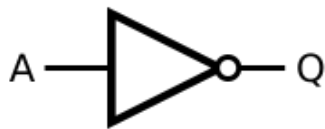


Figure 7: 1 input NOT Gate

| Input | Output |
|-------|--------|
| 0     | 1      |
| 1     | 0      |

Table 4: Truth Table of NOT Gate

## 4.2 Universal Gates

NAND and NOR gates are known as universal gates because they can be used to construct all other basic logic gates, including AND, OR, and NOT. This means that any digital logic circuit can be implemented using only NAND or only NOR gates. Their versatility makes them especially important in digital design and IC manufacturing, where minimizing the variety of components can lead to cost and space efficiency. Universal gates are commonly used in practice due to their functional completeness and availability in standard IC form. Understanding how to implement basic logic functions using NAND and NOR gates is a key step in mastering digital circuit design.

### 4.2.1 NAND Gate

The NAND gate is a fundamental digital logic gate that produces an output which is the inverse of the AND gate. It gives a low output (logic 0) only when all of its inputs are high; in all other cases, the output is high (logic 1). The name "NAND" stands for "NOT AND", reflecting its function. Due to its ability to replicate the behavior of all other basic gates when combined in various ways, the NAND gate is classified as a universal gate. It is widely used in digital electronics not only for its logical function but also for its versatility in circuit design. The standard symbol for a NAND gate is similar to an AND gate but includes a small circle (representing inversion) at the output.



Figure 8: 2 input NAND Gate

| A | B | Output |
|---|---|--------|
| 0 | 0 | 1      |
| 0 | 1 | 1      |
| 1 | 0 | 1      |
| 1 | 1 | 0      |

Table 5: Truth Table of NAND Gate

### 4.2.2 NAND Gate

The NAND gate is a fundamental digital logic gate that produces an output which is the inverse of the AND gate. It gives a low output (logic 0) only when all of its inputs are high; in all other cases, the output is high (logic 1). The name "NAND" stands for "NOT AND", reflecting its function. Due to its ability to replicate the behavior of all other basic gates when combined in various ways, the NAND gate is classified as a universal gate. It is widely used in digital electronics not only for its logical function but also for its versatility in circuit design. The standard symbol for a NAND gate is similar to an AND gate but includes a small circle (representing inversion) at the output.



Figure 9: 2 input NAND Gate

| A | B | Output |
|---|---|--------|
| 0 | 0 | 1      |
| 0 | 1 | 1      |
| 1 | 0 | 1      |
| 1 | 1 | 0      |

Table 6: Truth Table of NAND Gate

### 4.2.3 NOR Gate

The NOR gate is a digital logic gate that outputs high (logic 1) only when all of its inputs are low (logic 0). If any input is high, the output becomes low. It performs the logical NOT of the OR function, meaning it inverts the result of an OR gate. As a universal gate, the NOR gate can be combined in various ways to replicate the functions of other basic gates like AND, OR, and NOT. This makes it a versatile and valuable component in digital circuit design.



Figure 10: 2 input NOR Gate

| A | B | Output |
|---|---|--------|
| 0 | 0 | 1      |
| 0 | 1 | 0      |
| 1 | 0 | 0      |
| 1 | 1 | 0      |

Table 7: Truth Table of NOR Gate

## 4.3 Exclusive Gates

Exclusive gates include the XOR (Exclusive OR) and XNOR (Exclusive NOR) gates, which are used to compare binary inputs. These gates are especially important in arithmetic circuits, error detection systems, and digital comparators. While not considered universal gates, XOR and XNOR play a crucial role in logic designs where conditional logic and equality checking are required.

### 4.3.1 X-OR

The XOR (Exclusive OR) gate is a digital logic gate that outputs high (logic 1) only when its inputs are different. For two inputs, the output is high if one input is high and the other is low; if both inputs are the same (both 0 or both 1), the output is low (logic 0).



Figure 11: 2 input XOR Gate

| A | B | Output |
|---|---|--------|
| 0 | 0 | 0      |
| 0 | 1 | 1      |
| 1 | 0 | 1      |
| 1 | 1 | 0      |

Table 8: Truth Table of X-OR Gate

### 4.3.2 X-NOR

The XNOR (Exclusive NOR) gate is the logical complement of the XOR gate. It produces a high output (logic 1) only when its inputs are the same—either both 0 or both 1. If the inputs differ, the output is low (logic 0). The XNOR gate is typically represented by the XOR gate symbol with an added small circle (inversion bubble) at the output.



Figure 12: 2 input X-OR Gate

| A | B | Output |
|---|---|--------|
| 0 | 0 | 1      |
| 0 | 1 | 0      |
| 1 | 0 | 0      |
| 1 | 1 | 1      |

Table 9: Truth Table of NOR Gate

## 5 Overview of Logic Gates and ICs

| Gate Name | Logical Expression      | IC Number |
|-----------|-------------------------|-----------|
| AND       | $A \cdot B$             | 7408      |
| OR        | $A + B$                 | 7432      |
| NOT       | $\overline{A}$          | 7404      |
| NAND      | $\overline{A \cdot B}$  | 7400      |
| NOR       | $\overline{A + B}$      | 7402      |
| XOR       | $A \oplus B$            | 7486      |
| XNOR      | $\overline{A \oplus B}$ | 74266     |

Table 10: Logic Gates and their corresponding ICs (TTL – 74 series)

## 6 IC Internal Diagram

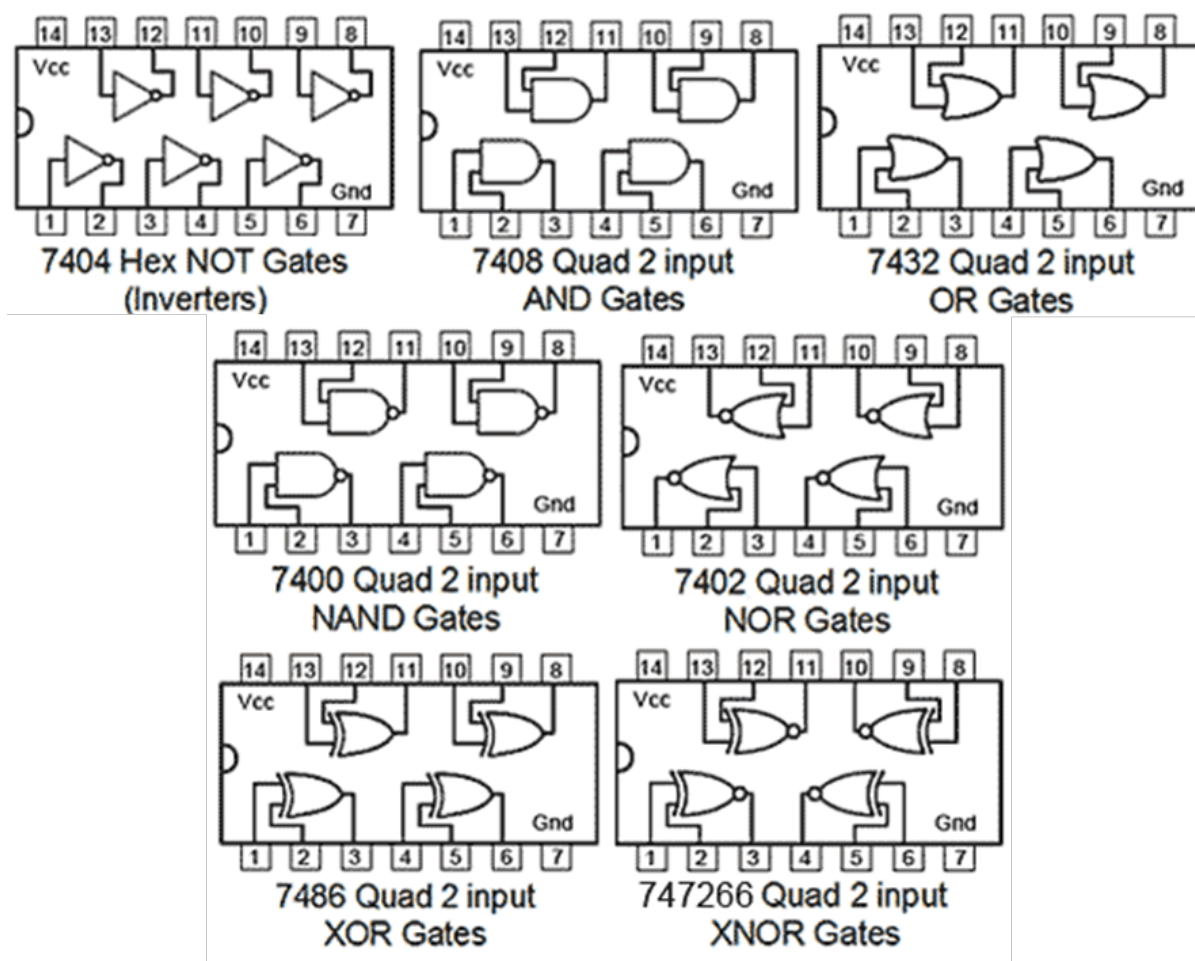


Figure 13: Internal Structure of ICs

## **7 Experiment 1: Introduction to Logic Gates and Their Truth Tables**

### **7.1 Objectives**

The objectives of this experiment are as follows:

- To study the basic logic gates: AND, OR, NOT, NAND, and NOR.
- To analyze the representation of these logic functions using truth tables, logic diagrams, and Boolean algebra.
- To observe and analyze the pulse response of various logic gates.

### **7.2 Tasks**

Today's lab task are as follows -

**7.2.1 Task 1**

Complete the logic gate symbol, IC number, truth table, and describe one key output behavior.

| Gate | Gate Symbol | IC Number | Truth Table |        |        | Characteristics of Output |
|------|-------------|-----------|-------------|--------|--------|---------------------------|
| AND  |             |           | A           | B      | Output |                           |
|      |             |           | 0           | 0      |        |                           |
|      |             |           | 0           | 1      |        |                           |
|      |             |           | 1           | 0      |        |                           |
|      |             |           | 1           | 1      |        |                           |
| OR   |             |           | A           | B      | Output |                           |
|      |             |           | 0           | 0      |        |                           |
|      |             |           | 0           | 1      |        |                           |
|      |             |           | 1           | 0      |        |                           |
|      |             |           | 1           | 1      |        |                           |
| NOT  |             |           | A           | Output |        |                           |
|      |             |           | 0           |        |        |                           |
|      |             |           | 1           |        |        |                           |
| NAND |             |           | A           | B      | Output |                           |
|      |             |           | 0           | 0      |        |                           |
|      |             |           | 0           | 1      |        |                           |
|      |             |           | 1           | 0      |        |                           |
|      |             |           | 1           | 1      |        |                           |
| NOR  |             |           | A           | B      | Output |                           |
|      |             |           | 0           | 0      |        |                           |
|      |             |           | 0           | 1      |        |                           |
|      |             |           | 1           | 0      |        |                           |
|      |             |           | 1           | 1      |        |                           |
| XOR  |             |           | A           | B      | Output |                           |
|      |             |           | 0           | 0      |        |                           |
|      |             |           | 0           | 1      |        |                           |
|      |             |           | 1           | 0      |        |                           |
|      |             |           | 1           | 1      |        |                           |
| XNOR |             |           | A           | B      | Output |                           |
|      |             |           | 0           | 0      |        |                           |
|      |             |           | 0           | 1      |        |                           |
|      |             |           | 1           | 0      |        |                           |
|      |             |           | 1           | 1      |        |                           |

Table 11: Task 1

### 7.2.2 Task 2

Implement the following circuit and fill up the truth table -

| Circuit Diagram                                           | Truth Table                                                                                                                                                                                                                                                                                                                 |        |   |        |   |   |  |   |   |  |   |   |  |   |   |  |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|---|--------|---|---|--|---|---|--|---|---|--|---|---|--|
| <p style="text-align: center;"><i>Circuit Diagram</i></p> | <table border="1" style="margin: auto; border-collapse: collapse;"> <thead> <tr> <th>A</th> <th>B</th> <th>Output</th> </tr> </thead> <tbody> <tr><td>0</td><td>0</td><td></td></tr> <tr><td>0</td><td>1</td><td></td></tr> <tr><td>1</td><td>0</td><td></td></tr> <tr><td>1</td><td>1</td><td></td></tr> </tbody> </table> | A      | B | Output | 0 | 0 |  | 0 | 1 |  | 1 | 0 |  | 1 | 1 |  |
| A                                                         | B                                                                                                                                                                                                                                                                                                                           | Output |   |        |   |   |  |   |   |  |   |   |  |   |   |  |
| 0                                                         | 0                                                                                                                                                                                                                                                                                                                           |        |   |        |   |   |  |   |   |  |   |   |  |   |   |  |
| 0                                                         | 1                                                                                                                                                                                                                                                                                                                           |        |   |        |   |   |  |   |   |  |   |   |  |   |   |  |
| 1                                                         | 0                                                                                                                                                                                                                                                                                                                           |        |   |        |   |   |  |   |   |  |   |   |  |   |   |  |
| 1                                                         | 1                                                                                                                                                                                                                                                                                                                           |        |   |        |   |   |  |   |   |  |   |   |  |   |   |  |

### 7.2.3 Task 3: Elevator Call System

On a certain floor, there are two call buttons:

- **Up button** (Input A) – outputs HIGH (1) when pressed.
- **Down button** (Input B) – outputs HIGH (1) when pressed.

The elevator controller can only process **one request at a time**. If exactly one button is pressed, the elevator is summoned to that floor. If both buttons are pressed simultaneously, the controller cannot process the conflicting requests, so **no call is sent**. If no button is pressed, naturally no call is sent.

- **Output = 1** → Elevator is summoned.
- **Output = 0** → No call (no button pressed or both pressed).

Design a logic circuit that implements this behavior. Determine the logic expression and identify the gate(s) required.

| Circuit Diagram | Truth Table                                                                                                                                                                                                                                                                                                                                             |                |                |        |   |   |  |   |   |  |   |   |  |   |   |  |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|----------------|--------|---|---|--|---|---|--|---|---|--|---|---|--|
|                 | <table border="1" style="margin: auto; border-collapse: collapse;"> <thead> <tr> <th>Up But-<br/>ton</th> <th>Down<br/>Button</th> <th>Output</th> </tr> </thead> <tbody> <tr><td>0</td><td>0</td><td></td></tr> <tr><td>0</td><td>1</td><td></td></tr> <tr><td>1</td><td>0</td><td></td></tr> <tr><td>1</td><td>1</td><td></td></tr> </tbody> </table> | Up But-<br>ton | Down<br>Button | Output | 0 | 0 |  | 0 | 1 |  | 1 | 0 |  | 1 | 1 |  |
| Up But-<br>ton  | Down<br>Button                                                                                                                                                                                                                                                                                                                                          | Output         |                |        |   |   |  |   |   |  |   |   |  |   |   |  |
| 0               | 0                                                                                                                                                                                                                                                                                                                                                       |                |                |        |   |   |  |   |   |  |   |   |  |   |   |  |
| 0               | 1                                                                                                                                                                                                                                                                                                                                                       |                |                |        |   |   |  |   |   |  |   |   |  |   |   |  |
| 1               | 0                                                                                                                                                                                                                                                                                                                                                       |                |                |        |   |   |  |   |   |  |   |   |  |   |   |  |
| 1               | 1                                                                                                                                                                                                                                                                                                                                                       |                |                |        |   |   |  |   |   |  |   |   |  |   |   |  |

## 8 Additional Task (Installing Proteus)

For future labs, you will need a software called Proteus. Please download and install it on your laptop or PC before the next lab. Here is the link to the file: [Proteus](#).